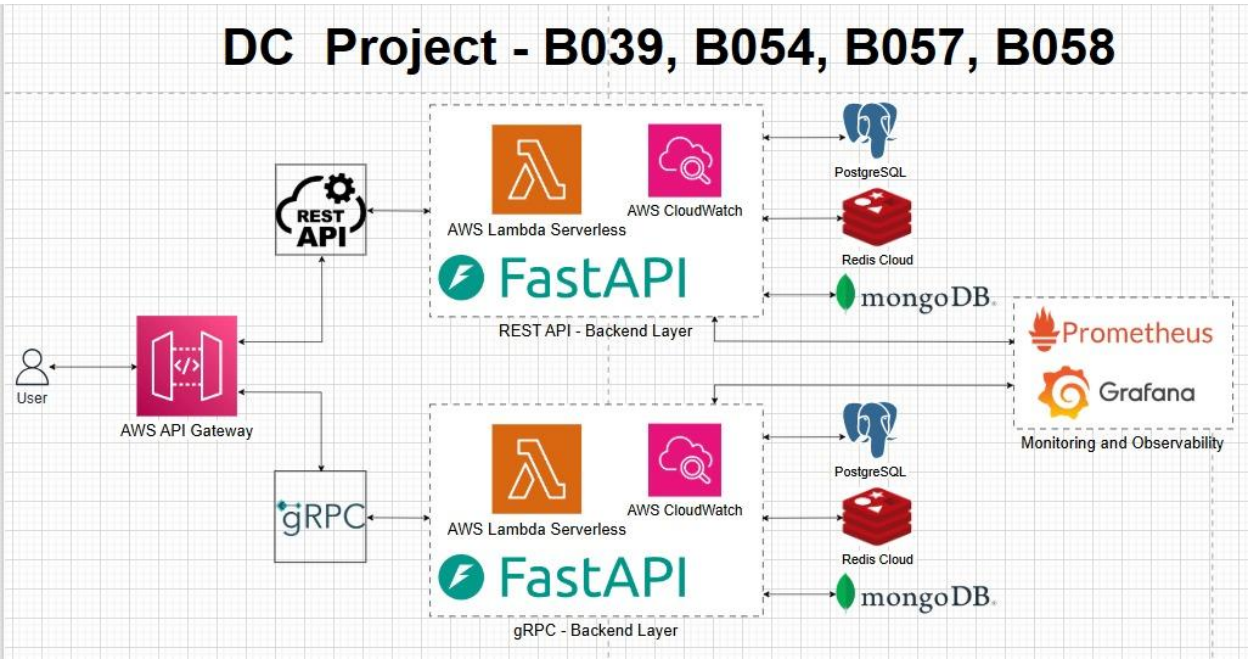
Progress Report – DC – B039, B054, B057, B058

1. Proposed System Architecture:

The architecture is designed as a Controlled Experimental Testbed. Its main goal is to separate communication protocol performance from application logic across data storage systems. By keeping the business logic the same and changing the transport layer and storage engine we can see where gRPC provides the most benefit.



The Service Layer: FastAPI and Asyncio

FastAPI is a web framework for building APIs with Python. It is based on Python type hints and uses the Starlette library for web parts and Pydantic for data parts, offering performance comparable to NodeJS and Go. It has the advantage of being able to support thousands of concurrent connections with a single thread event loop through the Asynchronous Server Gateway Interface (ASGI).

- Role in Testbed: We use FastAPI because its asynchronous capability is crucial for a concurrency test. FastAPI can handle thousands of wait states for database I/O operations without blocking a thread. This is an advantage because if there is a latency issue we know it is not due to a server bottleneck.
- REST Suite: This suite implements HTTP/1.1 endpoints using JSON for serialization. We aim to measure the Serialization Tax the text-parsing overhead that academic literature suggests can account for 10–40% of total response time under load.
- gRPC Suite: This suite generates a service that uses the Protocol Buffers (Protobuf) service. By using HTTP/2 features like HPACK header compression, which allows

multiple database requests to go over a TCP connection at once it removes the Head-of-Line (HOL) blocking commonly seen with REST.

The Database architecture: Three Paradigms

This architecture is designed to test if gRPC's efficiency is affected by database latency or if the binary serialization provides gains regardless of the storage backend.

PostgreSQL (Relational Storage)

PostgreSQL is an enterprise-class open-source RDBMS system. It supports SQL and JSON query languages. Is known for its architecture, dependability and data integrity.

- Role in Testbed: PostgreSQL represents the Heavy Compute tier. We use it to test gRPC under computational load involving complex schema mapping. Our goal is to determine if the binary format of Protobuf significantly reduces the CPU cycles spent on Object-Relational Mapping (ORM) overhead compared to JSON-based REST responses.

Redis (In-Memory Storage)

Redis is a source in-memory data structure store that can be used as a database, cache, message broker and streaming engine. It stores data in RAM. Offers sub-millisecond latency and can process millions of requests in a second.

- Role in Testbed: The speed of Redis is such that the database I/Os are no longer the performance bottleneck. The performance variable in this tier will be the network protocol (REST vs. GRPC). primary performance variable. This tier will demonstrate the "purest" performance benefits of the gRPC migration.

MongoDB (Document-Oriented Storage)

MongoDB is a document-oriented NoSQL database program that's cross-platform and source-available. It uses documents that resemble JSON as values and supports data types.

- Role in Testbed: MongoDB allows us to test how gRPC handles unstructured payloads compared to REST. This directly addresses the "Over/Under-fetching" issues mentioned in our literature review. We will measure if gRPC's strict typing and binary fragments can optimize the retrieval of large document sets efficiently than the self-descriptive but verbose JSON format.

The Observability Stack: Prometheus & Grafana

Prometheus (Metric Collection)

Prometheus is an open-source toolset for system monitoring and alerting. It follows a pull-based model, where it periodically "scrapes" services via an HTTP endpoint to obtain real-time metrics.

- Role in Testbed: We use Prometheus to gather the Four Golden Signals: Latency, Traffic, Errors and Saturation. Of conventional log data. We can focus on Tail Latency (p95/p99) by scraping these services every second. This provides the information needed for our migration approach that is validated by metrics.

Grafana (Visualization)

Grafana is a multiplatform open-source web tool for analytics and interactive visualization. It creates graphs, charts and alerts in a setting when linked to any supported data source, such as Prometheus.

- Role in Testbed: Grafana serves as the Visual Evidence layer of our project. We are building a Differential Dashboard that overlays the performance of REST and gRPC across all three databases. This allows us to visualize real-time performance deltas and identify the Saturation Point where the REST protocol begins to degrade compared to the gRPC implementation.

2. Justification of Design Choices: Deep Dive

Why the Three-Database architecture?

In a distributed system the Total Latency is the sum of Network Latency + Application Processing + Database I/O.

- Existing studies often only measure the network part.
- Our design tests the Amdahl's Law of APIs: if the database is the bottleneck does the protocol even matter? By testing Redis (fast) vs. Postgres (we provide a framework that tells engineers *when* a migration is actually worth the effort.

Why Prometheus & Grafana over manual logging?

Manual logging is insufficient for capturing Tail Latency (p99).

- In systems the mean (average) latency is not a good measure; the performance of a system is defined by its slowest 1% of requests.

- Prometheus allows us to set Phase-Gate Thresholds ("Only migrate to gRPC if the p99 remains below 50ms under 1000 requests/sec"). This directly addresses the "No Metric-Validated Migration Framework" gap we identified.

Why Local vs. Cloud (AWS)?

Local Docker environments lack the complexity of VPC Networking and Software-Defined Networking (SDN).

- Cloud environments introduce Jitter and variable network paths.
- Testing on AWS allows us to measure how TLS termination and mTLS handshakes (required by gRPC in production) impact performance, a variable often ignored in academic "local-only" benchmarks.

Why the Strangler Fig Pattern?

Migration is a business risk. The Strangler Fig allows us to wrap the existing REST-based CRUD services with an API Gateway.

- We can route 5% of traffic to the gRPC service while 95% stays on the stable REST service.
- If the Prometheus metrics show an increase in p99 latency for that 5% the system automatically rolls back. This Safety First approach is what makes our research practically applicable to real-world software engineering.

3. How Distributed Computing Concepts Are Applied

Concept	Application in Our Architecture
Transparency	The API Gateway makes the choice of protocol i.e., REST or gRPC transparent to the end-user.
Scalability	We have employed HTTP/2 Multiplexing to enable thousands of database queries to happen over a single connection.
Observability	We instrument the Four Golden Signals (Latency, Traffic, Errors, Saturation) into our CRUD services to monitor system health.
Consistency	Using .proto schema files ensures that our Postgres, Redis, and Mongo services have strict contracts, preventing "interface drift".
Tail Latency	We focus on p99 metrics (not just mean) to account for "The Tail at Scale" where the slowest DB request dictates total system performance.